

Prolog Program to Implement Towers of Hanoi (AI)

Aim

To write a Prolog program to solve the Towers of Hanoi puzzle for a given number of disks and display the sequence of moves required.

Objectives

- To understand recursive problem decomposition in Prolog.
- To move N-1 disks recursively before moving the largest disk.
- To display each move using format/2.
- To verify that the total number of moves equals $2^N - 1$.

Algorithm

- 1 Start.
- 2 Define the base case: move(1, From, To, _) directly moves disk 1 from From to To.
- 3 Define the recursive rule: move(N, From, To, Via) for $N > 1$.
- 4 Recursively move the top N-1 disks from From to Via (using To as auxiliary).
- 5 Move the Nth (largest) disk from From to To.
- 6 Recursively move the N-1 disks from Via to To (using From as auxiliary).
- 7 Display each individual disk movement as it happens.
- 8 Call move/4 with the initial number of disks, source, destination and auxiliary peg.
- 9 Stop.

Source Code (Prolog)

```
% Prolog program - Towers of Hanoi
move(1, From, To, _) :-
    format("Move disk 1 from ~w to ~w~n", [From, To]).
move(N, From, To, Via) :-
    N > 1,
    N1 is N - 1,
    move(N1, From, Via, To),
    format("Move disk ~w from ~w to ~w~n", [N, From, To]),
    move(N1, Via, To, From).
:- format("Towers of Hanoi solution for 3 disks:~n"),
    move(3, left, right, middle).
```

Output

```
Towers of Hanoi solution for 3 disks:
Move disk 1 from left to right
Move disk 2 from left to middle
Move disk 1 from right to middle
Move disk 3 from left to right
Move disk 1 from middle to left
Move disk 2 from middle to right
Move disk 1 from left to right
```

Result

The Prolog program successfully solved the Towers of Hanoi puzzle for 3 disks, producing the correct minimal sequence of 7 moves ($2^3 - 1$).